

Lecture 15: Training Neural Networks

Backpropagation · Regularization · Tuning

Recall: we can build a net, but not yet train one

Last lecture we built a network and ran a **forward pass**. The modest MNIST MLP had 109,386 weights.

To learn, we minimise the loss by **gradient descent** – which needs $\partial L / \partial w$ for *every* one of those weights.

Predict: how expensive is it to get all 109,386 gradients by nudging each weight and re-running the network?

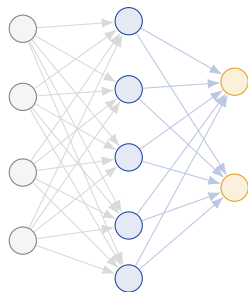
Recall: we can build a net, but not yet train one

Last lecture we built a network and ran a **forward pass**. The modest MNIST MLP had 109,386 weights.

To learn, we minimise the loss by **gradient descent** – which needs $\partial L / \partial w$ for *every* one of those weights.

Predict: how expensive is it to get all 109,386 gradients by nudging each weight and re-running the network?

One forward pass *per weight* $\rightarrow$ 109,386 forward passes for *one* gradient step. Hopeless. **Backpropagation** gets them all in essentially **one** extra pass.



$\sim 100k$ weights

Outline

Backpropagation

The training loop

Regularizing neural networks

Tuning and wrap-up

Backpropagation

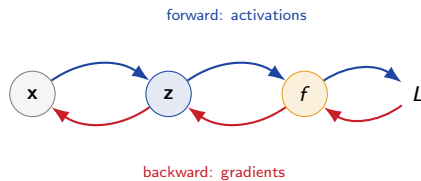
The chain rule, applied backward once, with every shared piece reused.

The idea: chain rule, backward, reuse

The loss is a **composition**: input $\rightarrow$ hidden $\rightarrow$ output $\rightarrow$ loss. The chain rule turns $\partial L / \partial w$ into a product of local derivatives along the path from w to L .

Compute these products **from the output backward**, and every layer reuses the gradient already computed for the layer above it. That reuse is the whole trick – one backward sweep, not one per weight.

Two sweeps per step: a **forward** pass caches activations; a **backward** pass turns them into gradients.



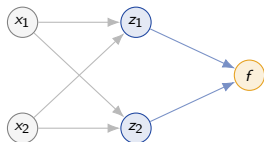
Forward pass: cache what the backward pass will need

Reuse the Lecture-14 net: 2 inputs,
2 sigmoid hidden, 1 sigmoid output.
Input $\mathbf{x} = (2, -1)$, target $y = 1$.

$$\mathbf{w}_1 = (0.5, -0.5), b_1 = 0$$

$$\mathbf{w}_2 = (-0.3, 0.8), b_2 = 0.1$$

$$\mathbf{u} = (1.0, -1.0), c = 0.2$$



Cache every intermediate (we need them going back):

$$z_{1,\text{in}} = 0.5(2) + (-0.5)(-1) + 0 = 1.5, \quad z_1 = \sigma(1.5) = 0.82$$

$$z_{2,\text{in}} = -0.3(2) + 0.8(-1) + 0.1 = -1.3, \quad z_2 = \sigma(-1.3) = 0.21$$

$$f_{\text{in}} = 1.0(0.82) - 1.0(0.21) + 0.2 = 0.80, \quad f = \sigma(0.80) = 0.69$$

Cross-entropy loss (target $y = 1$):

$$L = -(y \log f + (1-y) \log(1-f)) = -\log(0.69) = 0.37$$

The output gradient is just $f - y$

Before the numbers, one clean fact. With a **sigmoid output** and **cross-entropy** loss, the gradient at the output pre-activation collapses:

$$\begin{aligned}\frac{\partial L}{\partial f} &= -\frac{y}{f} + \frac{1-y}{1-f}, & \frac{\partial f}{\partial f_{\text{in}}} &= f(1-f) \\ \frac{\partial L}{\partial f_{\text{in}}} &= \left(-\frac{y}{f} + \frac{1-y}{1-f}\right) f(1-f) = -(y(1-f) - (1-y)f) = \boxed{f - y}\end{aligned}$$

The same “residual” you already know. Linear regression, logistic regression, and now a neural net: the output-layer gradient is **prediction** – **target**. Here $f - y = 0.69 - 1 = -0.31$.

Backward pass I: the output-layer weights

Call the output signal

$$\delta_{\text{out}} = \frac{\partial L}{\partial f_{\text{in}}} = f - y = -0.31.$$

Each output weight's gradient is δ_{out} times the input on that edge (z_j from the forward cache):

$$\frac{\partial L}{\partial u_1} = \delta_{\text{out}} z_1 = -0.31(0.82) = -0.25$$

$$\frac{\partial L}{\partial u_2} = \delta_{\text{out}} z_2 = -0.31(0.21) = -0.07$$

$$\frac{\partial L}{\partial c} = \delta_{\text{out}} = -0.31$$

Pattern. Every weight gradient has the form

$$\frac{\partial L}{\partial(\text{weight})} = (\text{signal at its output end}) \times (\text{activation})$$

The forward cache supplies the second factor; the backward pass supplies the first.

Negative gradients $\Rightarrow$ gradient descent will *increase* u_1, u_2, c , pushing f up toward the target $y = 1$.
Good.

Backward pass II: reach the hidden-layer weights

Push the signal back through the output weight and the hidden sigmoid. For hidden neuron j , with $\sigma'(z_{\text{in}}) = z(1 - z)$:

$$\delta_j = \delta_{\text{out}} u_j \underbrace{z_j(1 - z_j)}_{\sigma'(z_{j,\text{in}})}$$

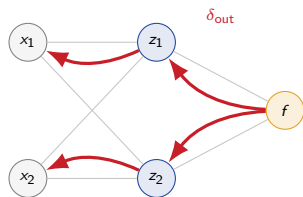
$$\delta_1 = -0.31(1.0)(0.82)(0.18) = -0.046$$

$$\delta_2 = -0.31(-1.0)(0.21)(0.79) = +0.052$$

Then, same pattern as before ($\delta_j \times \text{input } x_i$):

$$\frac{\partial L}{\partial \mathbf{w}_1} = \delta_1 \mathbf{x} = (-0.092, 0.046)$$

$$\frac{\partial L}{\partial \mathbf{w}_2} = \delta_2 \mathbf{x} = (0.104, -0.052)$$



The signal flows right to left; each node scales it by a local derivative.

Why backprop is cheap: each δ computed once

Look again at the hidden-layer gradients. *Both* weights feeding neuron 1 reused the **same** signal δ_1 :

$$\frac{\partial L}{\partial \mathbf{w}_1} = \delta_1 \mathbf{x} = \left(\underbrace{\delta_1 x_1}_{\partial L / \partial w_{11}}, \underbrace{\delta_1 x_2}_{\partial L / \partial w_{12}} \right)$$

And δ_1 was itself built from δ_{out} – the signal one layer above.

So each layer computes its δ **once** from the layer above, then every weight into that layer is just $\delta \times \text{input}$. Nothing is recomputed.

Cost, side by side (5 weights here):

- **Naive:** nudge each weight, re-run forward $\rightarrow$ 5 forward passes.
- **Backprop:** one backward sweep reuses the shared δ 's $\rightarrow$ *one* pass gives all 5 gradients.

On the MNIST net that is 109,386 forward passes vs. **one**. The reused δ is the entire reason training is affordable.

One step of gradient descent, and the loss drops

We now have a gradient for every weight. Update each by $w \leftarrow w - \eta \partial L / \partial w$ with learning rate $\eta = 0.5$:

$$u_1 : 1.0 - 0.5(-0.25) = 1.13$$

$$u_2 : -1.0 - 0.5(-0.07) = -0.97$$

$$c : 0.2 - 0.5(-0.31) = 0.35$$

$$\mathbf{w}_1 : (0.5, -0.5) - 0.5(-0.092, 0.046)$$

$$\mathbf{w}_2 : (-0.3, 0.8) - 0.5(0.104, -0.052)$$

Re-run the forward pass with the updated weights:

$$f : 0.69 \rightarrow 0.75, \quad L : 0.37 \rightarrow 0.28$$

The prediction moved **toward the target** ($y = 1$) and the loss **fell**. That is one training iteration. Real training does this for *thousands* of mini-batches.

Sanity check: is your backprop even correct?

Backprop is easy to get subtly wrong. Check it against a **numerical gradient** from the definition of a derivative:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}, \quad \varepsilon \approx 10^{-5}$$

- ▶ Nudge *one* weight up and down, re-run the forward pass, take the difference.
- ▶ Compare to your analytic (backprop) gradient – they should agree to $\sim 10^{-7}$.
- ▶ If they do not, your backward pass has a bug.

Debug only. This costs *two* forward passes *per weight* – the very cost backprop avoids. Use it on a tiny net to validate the code, never as the trainer.

Backprop, vectorised: the same rule for a whole network

By hand for 5 weights; as matrices for a million. Let $\delta^{(l)}$ be the signal at layer l 's pre-activations. Two rules, applied from the last layer backward:

$$\delta^{(L)} = \nabla_a L \odot \varphi'(\mathbf{z}^{(L)}), \quad \delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \varphi'(\mathbf{z}^{(l)})$$
$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

- ▶ $\odot$ is elementwise; φ' is the activation's derivative (e.g. $z(1-z)$ for sigmoid).
- ▶ Exactly the scalar steps you just did – “signal $\times$ local derivative”, “signal $\times$ input” – now one matrix multiply per layer.

Why deep sigmoid nets barely learn

The recursion $\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \varphi'(\mathbf{z}^{(l)})$ multiplies by φ' at every layer on the way back.

- Sigmoid: $\varphi' \leq 0.25$. After k layers the gradient is scaled by $\leq 0.25^k$ – it **vanishes exponentially** with depth. Early layers get almost no signal.
- ReLU: $\varphi' = 1$ for $z > 0$ – the gradient passes through undiminished.

$0.25^{10} \approx 10^{-6}$. A 10-layer sigmoid net shrinks the gradient a *millionfold* before it reaches the first layer – so the first layer effectively stops learning.

This is the **vanishing-gradient** problem we flagged in L14, now visible in the math. Three fixes, all coming up: **ReLU**, careful **initialization**, and **BatchNorm**.

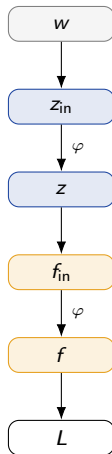
Backprop is the chain rule on a computational graph

Everything from input to loss is one **computational graph**: each intermediate (z_{in} , z , f_{in} , f , L) is a **node** – a function of its parents. The forward pass just evaluates the nodes top to bottom.

A weight w affects L only *along the path* from w to L . The chain rule says: multiply the **local derivative** on each edge of that path.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial f_{\text{in}}} \frac{\partial f_{\text{in}}}{\partial z} \frac{\partial z}{\partial z_{\text{in}}} \frac{\partial z_{\text{in}}}{\partial w}$$

That is exactly the product you formed by hand – now named.



forward evaluates down; the chain rule multiplies edge-derivatives back up.

Reverse mode: one sweep, any architecture

When a node feeds **several** children (a hidden unit into many outputs), its gradient is the **sum over all outgoing paths** – the multivariate chain rule:

$$\frac{\partial L}{\partial z} = \sum_k \frac{\partial L}{\partial f_{in,k}} \frac{\partial f_{in,k}}{\partial z}$$

Reverse-mode autodiff visits each node *once*, from L backward, storing $\partial L / \partial(\text{node})$ as it goes – exactly the cached δ 's, now on the whole graph.

Why this matters. The rule is stated on the *graph*, not on “layers”. So the same engine differentiates **convolutions, attention, residual connections** – any composition of differentiable ops – with no new math. That is what makes deep-learning frameworks general.

You built the graph for one net by hand; autograd builds it for *any* net automatically. Next slide.

You will never write this by hand: autograd

Frameworks record every operation of the forward pass into a graph, then replay it backward – running exactly the two rules from two slides ago – when you call `.backward()`.

```
loss = loss_fn(model(x), y)    # forward: build the graph
loss.backward()                # backward: backprop
```

You still need to *understand* backprop – to read training curves, debug dead layers, and know why ReLU and good init matter – but you will call `loss.backward()`, not differentiate by hand.

The training loop

Mini-batches, an optimizer, and the one hyperparameter you always touch.

Batch, stochastic, mini-batch

Gradient descent needs the gradient of the loss over *some* data. How much?

- ▶ **Batch GD:** use all n examples per step. Accurate gradient, but one slow step; infeasible for large data.
- ▶ **Stochastic GD:** one example per step. Cheap, very noisy.
- ▶ **Mini-batch GD:** a small batch (e.g. 32–256) per step. The standard – good gradient estimate, hardware-friendly, and the noise even helps escape bad spots.

One **epoch** = one full pass over the training set = many mini-batch steps. You train for many epochs. Same gradient-descent idea as regression – just fed data in batches.

The training loop, in full

Everything in this lecture is these few lines. The rest is choosing what goes in them.

```
for epoch in range(n_epochs):  
    for xb, yb in loader:           # mini-batches  
        optimizer.zero_grad()      # reset grads  
        out = model(xb)            # forward pass  
        loss = loss_fn(out, yb)  
        loss.backward()             # backprop  
        optimizer.step()            # update weights
```

The full runnable version – data loading, evaluation, saving – lives in the practical, not here.

Which optimizer? Reuse the optimization module

`optimizer.step()` hides a real choice. Plain mini-batch **SGD** works, but two refinements are near-universal:

- ▶ **Momentum:** accumulate a running “velocity” of past gradients – rolls through small bumps and noise instead of zig-zagging.
- ▶ **Adam:** keep a *per-parameter* adaptive step size (roughly, momentum + per-weight scaling). The common default.

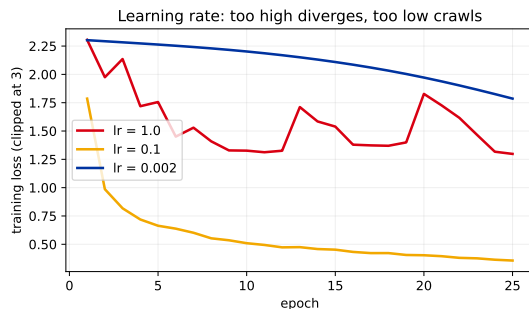
This is a topic you have already covered. For how momentum and Adam actually work, see the course Optimization module – “10 Optim: Momentum + First Order Methods” (and “09 Optim: Prerequisites and Gradient Descent”). **Practical default: start with Adam.**

The one knob you always touch: learning rate

The learning rate η scales every step.

- ▶ **Too high:** steps overshoot; the loss oscillates or diverges.
- ▶ **Too low:** steps are tiny; training crawls and may stall.
- ▶ **Right:** steady, fast decrease.

Predict: which of the three curves is too high, too low, and just right?



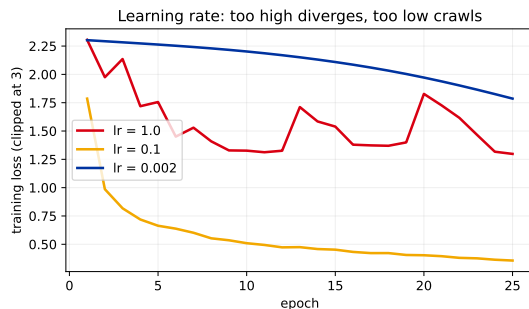
The one knob you always touch: learning rate

The learning rate η scales every step.

- ▶ **Too high:** steps overshoot; the loss oscillates or diverges.
- ▶ **Too low:** steps are tiny; training crawls and may stall.
- ▶ **Right:** steady, fast decrease.

Predict: which of the three curves is too high, too low, and just right?

Answer: $lr = 1.0$ (red) stalls high – steps overshoot; $lr = 0.002$ (blue) barely moves; $lr = 0.1$ (orange) is the steady, fast drop. Learning rate is the first thing to tune.



Beyond a fixed rate: learning-rate schedules

A single fixed η is a compromise: big enough to move early, small enough to settle late. **Schedules** vary η over training to get both.

- ▶ **Decay:** start larger, shrink over time (step, linear, or exponential) – fast progress first, fine-tuning later. The standard.
- ▶ **Warmup:** start *tiny* for a few hundred steps, then ramp up – stops a fresh, unstable net from diverging on the first big step.

- ▶ **Cyclical / cosine:** let η rise and fall in cycles – the bumps can kick the optimizer out of saddles and poor minima.

Practical: a fixed rate with Adam is a fine start; add *decay* (or warmup + cosine, the modern default for big models) once you are tuning for the last few points.

A word on the loss landscape

With 100k weights the loss surface is wildly **non-convex** – countless local minima and saddle points. Classical optimization would despair.

- ▶ In practice, mini-batch SGD/Adam finds *good enough* minima reliably; in high dimensions most are about equally good, and the noise helps escape bad ones.
- ▶ *Why* this works so well is still an active research question – we take it as an empirical fact and move on.

Contrast with linear/logistic regression (convex: one global minimum, L1). Depth buys power at the cost of a messy landscape.

Saddles and cliffs – and how we cope

Two obstacles appear once the landscape is non-convex and high-dimensional:

Saddle points dominate. At a flat spot, a true minimum needs *every* curvature direction to bend **up**. With thousands of dimensions that is exponentially unlikely, so almost every flat spot is a **saddle** (up in some directions, down in others), not a trap.

Good news: the **noise** in mini-batch SGD nudges you off saddles that a noiseless method would sit on forever.

None of this existed in the convex world of linear/logistic regression (L11) – the price of a powerful, non-convex model.

Cliffs / exploding gradients. Deep and recurrent nets can have near-vertical walls in the loss surface; one step there flings the weights far and undoes training.

Fix: gradient clipping – cap the gradient's norm at a threshold h :

$$\text{if } \|\nabla\| > h : \quad \nabla \leftarrow \frac{h}{\|\nabla\|} \nabla$$

Regularizing neural networks

A 100k-parameter model overfits eagerly. Five ways to hold it back.

Nets overfit eagerly

A network has more than enough capacity to **memorise** the training set – driving train loss to zero while validation loss climbs. Everything in this section fights that gap.

- ▶ Same **bias-variance** story as the rest of the course (L01d / [06]) – nothing new conceptually, just a very high-variance model.
- ▶ The tools: **dropout**, **weight decay**, **data augmentation**, good **initialization** + **BatchNorm**, and **early stopping**.

We will read the tell-tale train-vs-validation curves at the end of the section.

Dropout: train a crowd of subnetworks

Predict: you delete a random 50% of the hidden neurons on *every* mini-batch. Does that help or hurt?

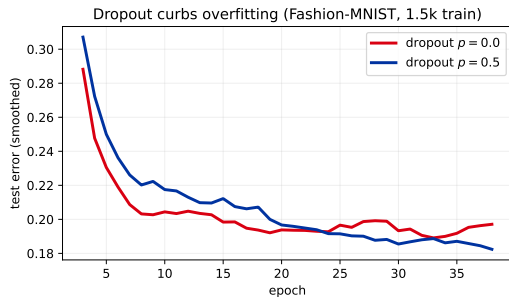
Dropout: train a crowd of subnetworks

Predict: you delete a random 50% of the hidden neurons on *every* mini-batch. Does that help or hurt?

It **helps**. Each step trains a different random subnetwork, so no neuron can rely on a specific partner – they must learn robust, redundant features. At test time you use all neurons: an implicit **ensemble average** over exponentially many subnetworks.

Callback: this is **bagging** (ch4) done inside one network. Typical rate 0.2–0.5.

In practice: PyTorch uses *inverse* dropout – it scales the surviving activations by $1/(1-p)$ *during training*, so nothing changes at test time.



No dropout ($p=0$) overfits; a little dropout lowers test error.

Weight decay: you already know this one

Add a penalty on the size of the weights to the loss – exactly **Ridge** from L01b, now called weight decay:

$$L_{\text{reg}} = L + \frac{\lambda}{2} \|\mathbf{W}\|_2^2 \quad \implies \quad w \leftarrow w - \eta \left(\frac{\partial L}{\partial w} + \lambda w \right)$$

- ▶ Every step shrinks each weight a little toward zero – hence “decay”.
- ▶ Smaller weights $\Rightarrow$ a smoother function $\Rightarrow$ less overfitting.
- ▶ In PyTorch it is one argument: `optimizer = Adam(..., weight_decay=1e-4)`.

Same L2 penalty as Ridge regression – the network is still “a stack of regressions”, so the regressions’ regularizer still applies.

Data augmentation: free training data

Overfitting is a ratio: model capacity over amount of data. You cannot easily shrink a net, but you can **grow the data** – cheaply and for free.

Idea: apply *label-preserving* transformations to each example, so the model sees many variants of the same thing.

- ▶ Images: flips, small rotations, crops, brightness, noise.
- ▶ Audio: time-stretch, pitch shift, background noise.

Label-preserving is the catch. Rotate a handwritten 6 too far and it becomes a 9 – now the label is wrong. Augmentations must respect what the label means.

Augmentation is strongest exactly where nets are strong: raw perceptual data with known invariances.

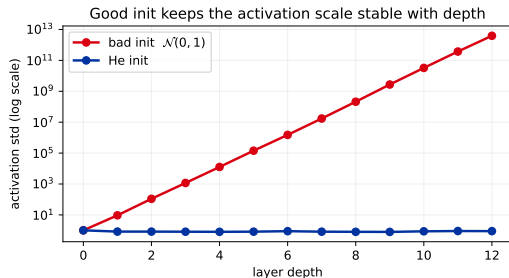
Initialization: give gradients a fighting start

Where you *start* the weights decides whether signal survives depth. Too large and activations explode; too small and they vanish – before any learning happens.

Fix: scale the initial weights by the fan-in so variance is preserved layer to layer.

- ▶ **He init** (ReLU): $w \sim \mathcal{N}(0, 2/n_{\text{in}})$.
- ▶ **Xavier/Glorot** (tanh/sigmoid):
 $w \sim \mathcal{U}(\pm \sqrt{6/(n_{\text{in}} + n_{\text{out}})})$.

Frameworks default to sensible init, but this is the first thing to check when a deep net will not train.



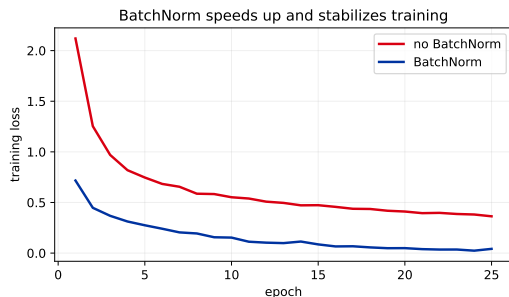
Bad init: activation scale blows up with depth. He init: stable.

BatchNorm: normalize activations mid-network

Even with good init, activation statistics drift as training proceeds. **Batch normalization** standardizes each layer's pre-activations z over the mini-batch B , then rescales with two *learned* parameters:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \text{out} = \gamma \hat{z} + \beta$$

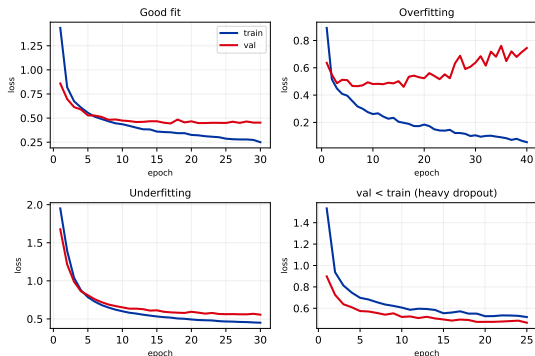
- ▶ μ_B, σ_B^2 : the batch mean/variance; keeps signals in range $\Rightarrow$ faster training, higher learning rates, mild regularization noise.
- ▶ γ, β let the net rescale/shift back if it needs to.
- ▶ One layer in PyTorch:
`nn.BatchNorm1d(width)`.



Same net and optimizer; BatchNorm reaches low loss far sooner.

Read your training curves

Plot train *and* validation loss every epoch – the single most useful diagnostic.



- **Good:** both fall, validation a little above train.
- **Overfitting:** train keeps falling, validation turns *up* – stop at the valley (**early stopping**).
- **Underfitting:** both stay high – too small / too much regularization / too few epochs.
- **Val < train:** usually heavy dropout (train loss is measured with noise on) – or a leak. Investigate.

Tuning, and putting it together

Which knob for which symptom – and what is genuinely new here.

Hyperparameters: symptom → fix

Knob		If underfitting	If overfitting
Model (depth/width)	size	increase capacity	decrease, or add regularization
Learning rate		raise (or use Adam)	usually not the cause
Epochs		train longer	fewer / early stopping
Dropout		lower it	raise it (0.2–0.5)
Weight decay		lower it	raise it
Data augmentation		(not the issue)	add more

First diagnose *under- vs over-fitting* from the curves, *then* reach for the matching knob. Do not turn all of them at once.

Tuning machinery is not new

Choosing these hyperparameters is a search problem you already have tools for:

- ▶ **Grid / random search**, and **Optuna** for smarter search – all from lecture [08]. Nothing NN-specific about the search itself.
- ▶ **Early stopping** doubles as the cheapest tuner: it picks the epoch count for you, for free, every run.
- ▶ Neural nets just have *more* knobs and cost *more* per trial – so lean on early stopping and sensible defaults before launching a big sweep.

Sensible starting recipe: Adam, ReLU, a little dropout, weight decay 10^{-4} , early stopping on validation loss. Tune from there.

Recap

- ▶ **Backprop** = the chain rule applied backward once, reusing shared terms: “signal $\times$ local derivative” per layer. Output signal is $f - y$, the same residual as regression.
- ▶ Vectorised, it is two matrix rules per layer; `loss.backward()` runs them for you. Sigmoid’s small φ' is why deep nets need **ReLU / good init / BatchNorm**.
- ▶ **Training loop**: mini-batch SGD, an optimizer (**start with Adam**; details in the Optim module), and the **learning rate** as the key knob.
- ▶ **Regularize**: dropout (ensemble), weight decay (= Ridge), augmentation, init+BatchNorm, early stopping. **Read the train-vs-val curves** to know which.
- ▶ **Tuning** reuses grid/random/Optuna ([08]); nets just have more knobs.

Next: the practical – train an MLP end to end in PyTorch. Later, the deep-learning chapter (CNNs, RNNs, transformers) builds on exactly this training machinery.